

Improving Autonomous Robot Localization using GPS and IMU Data Fusion: Progress Report 2

Steffen Ullmann
Florida Polytechnic University
Lakeland, FL, USA
sullmann3121@floridapoly.edu

Abstract—This report covers the Week 2 progress of the GPS/IMU sensor fusion project. All planned Week 2 objectives have been completed: full 6-DOF IMU data acquisition (accelerometer and gyroscope) was verified, and ROS2 Humble was successfully installed on the Rock5C compute board. Additionally, the first two ROS2 sensor nodes which were originally planned for Week 3 were implemented and verified, placing the project one week ahead of schedule.

Index Terms—GPS, IMU, ROS2, Sensor Fusion, Autonomous Robot, Localization

I. INTRODUCTION

This project fuses GPS and IMU measurements on an RC-car platform to improve localization accuracy beyond what a standalone u-blox 7 GPS receiver provides [1], [2]. A full system description and Week 1 results are in Progress Report 1. Week 2 targeted completing IMU sensor reads and establishing a ROS2 environment; both goals were achieved, and the first ROS2 sensor nodes were implemented ahead of schedule.

II. PROGRESS TO DATE

IMU full 6-DOF reads (test_imu_i2c.py): Building on the temperature and identity reads from Week 1, accelerometer and gyroscope output registers are now read in two sequential I2C burst transfers. A single transfer starting at ACCEL_XOUT_H retrieves six consecutive bytes covering all three axes, avoiding separate per-axis reads. The same burst strategy is applied to GYRO_XOUT_H. Raw 16-bit unsigned values are converted to signed integers via two's-complement arithmetic before scaling. Full-scale ranges are configured by writing to ACCEL_CONFIG (± 4 g, LSB sensitivity 8192 LSB/g) and GYRO_CONFIG (± 500 °/s, LSB sensitivity 65.5 LSB/(°/s)); each write is followed by a read-back to confirm the register value. Output is printed at 100 Hz.

ROS2 Humble installation: The Rock5C (RK3588S, AArch64) is not supported by the standard Debian repository, so a pre-built ros2_humble.tar.gz archive was extracted to /home/radxa/ros2_humble [3]. The archive contains hardcoded absolute paths, so the extraction target must match exactly. Four additional system packages were required: libspidev-dev, python3-packaging, python3-dev, and python3-netifaces. The Python module lark was installed via pip to enable ros2-launch. The setup script is sourced from ~/ros2_humble/install/setup.bash.

robot_sensors ROS2 package: A new ament_python package was created containing two publisher nodes and a shared helper module. A central settings.py holds all hardware constants (I2C bus, sensor addresses, full-scale ranges, gravity constant). Architecture and data flow are shown in Fig. 1.

- **imu_publisher_node:** Initializes the MPU-6050 (scale configuration with verification) and publishes sensor_msgs/Imu on /imu/data_raw at 100 Hz. Linear acceleration is converted from g to m/s²; angular velocity is converted from °/s to rad/s as required by the ROS2 message convention. Orientation and all covariance fields are set to -1 (unknown).
- **gps_publisher_node:** Reads NMEA sentences from /dev/ttyACM0 and publishes sensor_msgs/NavSatFix on /gps/fix when a \$GNGGA/\$GPGGA sentence with a valid fix is received. A no-fix condition is reported via status.status = -1.
- **Launch file:** sensor_launch.xml starts both nodes with a single ros2-launch command.

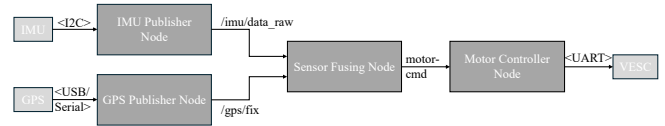


Fig. 1. Planned ROS2 node and topic architecture. Shaded boxes left of centre (IMU Publisher Node, GPS Publisher Node) are implemented; the Sensor Fusing Node and Motor Controller Node are planned for Weeks 3–4.

III. PLANNED WORK

The project is one week ahead of schedule, having completed the ROS2 sensor nodes originally targeted for Week 3.

Sensor fusion (Week 3): Implement and tune an Extended Kalman Filter (EKF) fusing /imu/data_raw and /gps/fix. The UKF remains a fallback if the EKF linearization proves insufficient. Localization accuracy will be quantified by comparing GPS-only against fused position estimates.

Localization evaluation and trajectory control (Weeks 4–5): Field experiments will measure position error. If schedule allows, a Pure Pursuit or Stanley trajectory controller will be implemented as a secondary goal.

The updated project timeline is given in Section IV.

IV. APPENDIX: UPDATED PROJECT TIMELINE

TABLE I

PROJECT TIMELINE. WEEKS 1 AND 2 COMPLETE; EARLY COMPLETION OF ROS2 SENSOR NODES PROVIDES ONE WEEK OF SCHEDULE MARGIN.

Week	Task	Status
1	Hardware assembly, sensor interface verification	✓
2	IMU accel/gyro reads, ROS2 environment setup on Rock5C	✓
2 (extra)	ROS2 GPS & IMU publisher nodes, robot_sensors package	✓ (ahead)
3	EKF/UKF sensor fusion implementation and initial tuning	
4	Localization accuracy evaluation, fusion parameter refinement	
5	Field testing, trajectory controller (<i>if schedule allows</i>), final report	

REFERENCES

- [1] "NEO-7 u-blox 7 GPS/GNSS Modules Data Sheet." 2013. [Online]. Available: [https://content.u-blox.com/sites/default/files/products/documents/NEO-7_DataSheet_\(UBX-13003830\).pdf](https://content.u-blox.com/sites/default/files/products/documents/NEO-7_DataSheet_(UBX-13003830).pdf)
- [2] U.S. Space Force, "GPS Accuracy." [Online]. Available: <https://www.gps.gov/>
- [3] S. Ullmann, "GPS Autonomous Robot – Project Repository." [Online]. Available: https://github.com/SteffEng-lab/gps_autonomous_robot